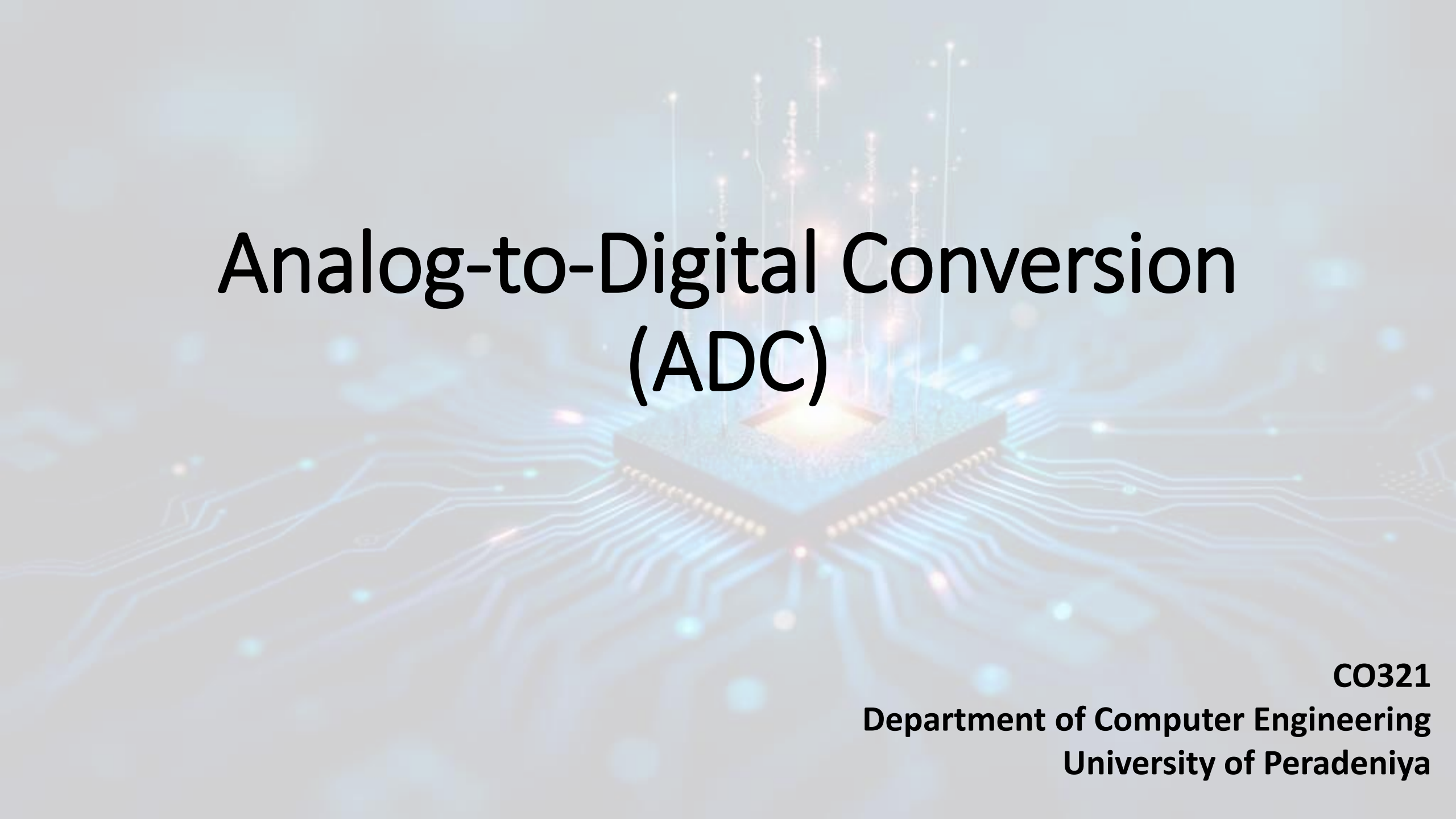


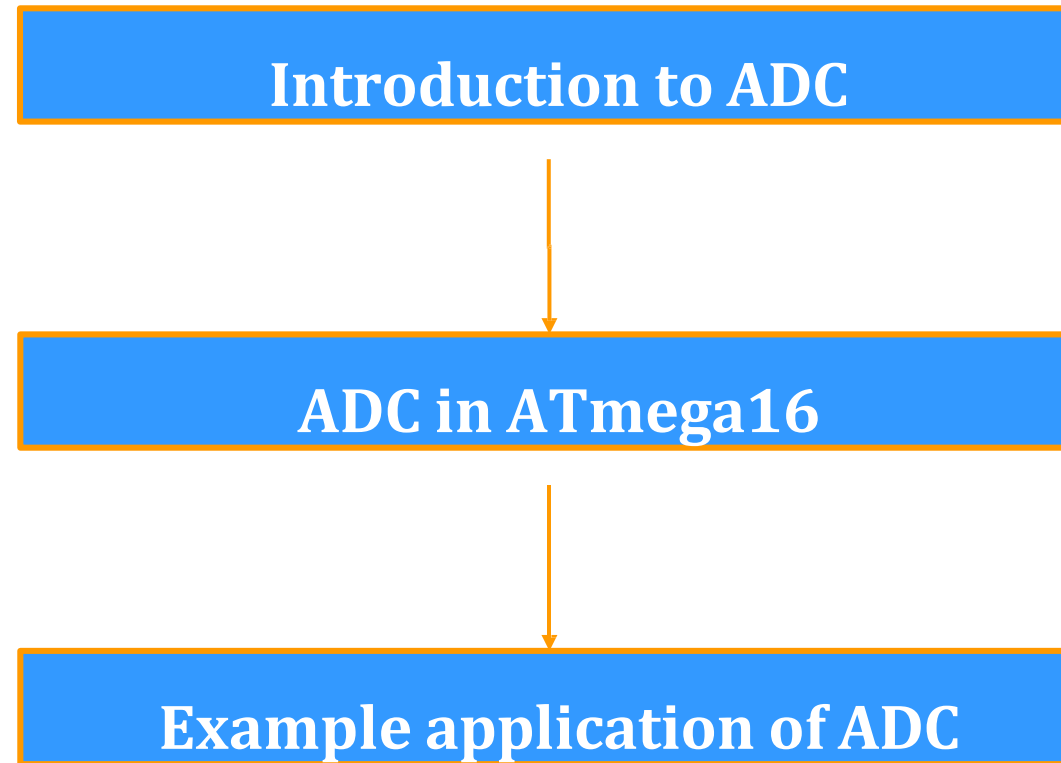
Analog-to-Digital Conversion (ADC)

A glowing microchip with circuit traces and data lines, symbolizing digital technology.

CO321

**Department of Computer Engineering
University of Peradeniya**

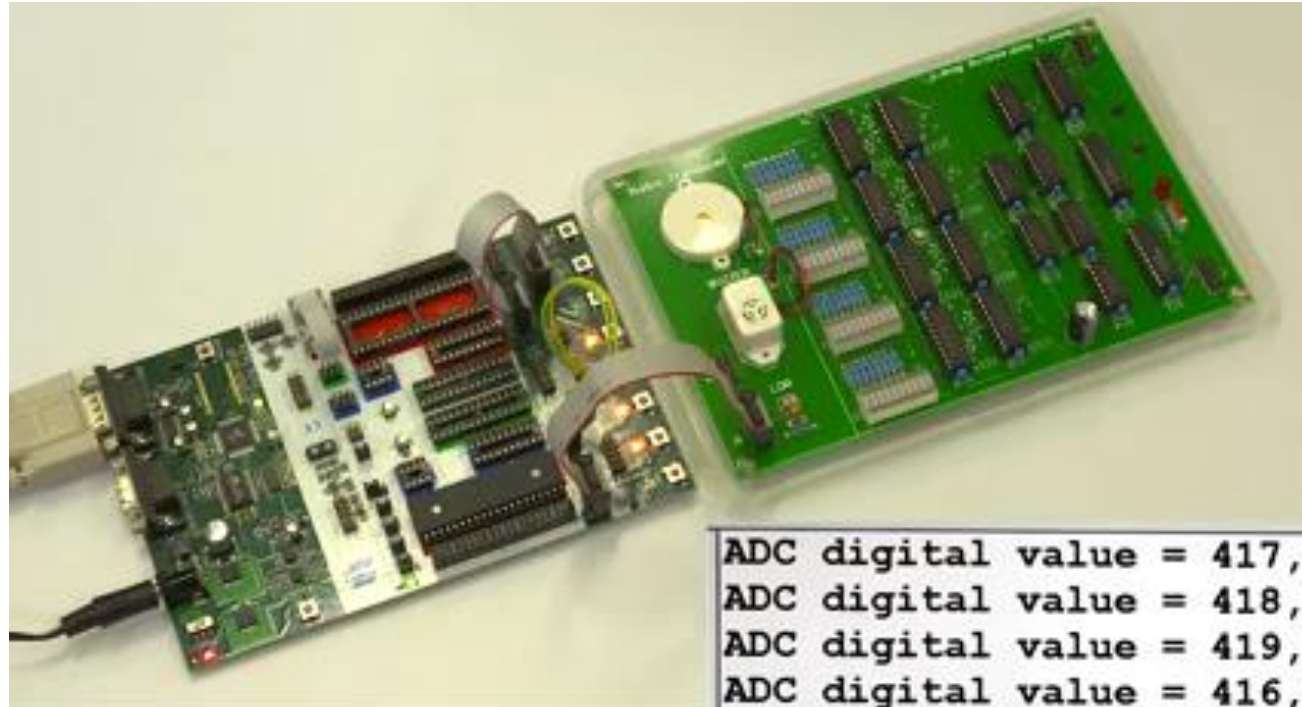
Lecture sequence



Introduction to A-to-D conversion

- An ADC samples an analogue signal at discrete times, and converts the sampled signal to a digital form.
- Used with transducers, ADCs allow us to monitor real-world inputs and perform control operations based on these inputs.
- Hardware Reality:
 - Past: Dedicated external ICs (e.g., ADC0804 - 8-bit Successive Approximation, Maxim104: 8-bit, flash type).
 - Present: Integrated directly into the MCU silicon (e.g., ATmega16, STM32, ESP32) to save cost, board space, and power.

ADC Demonstration

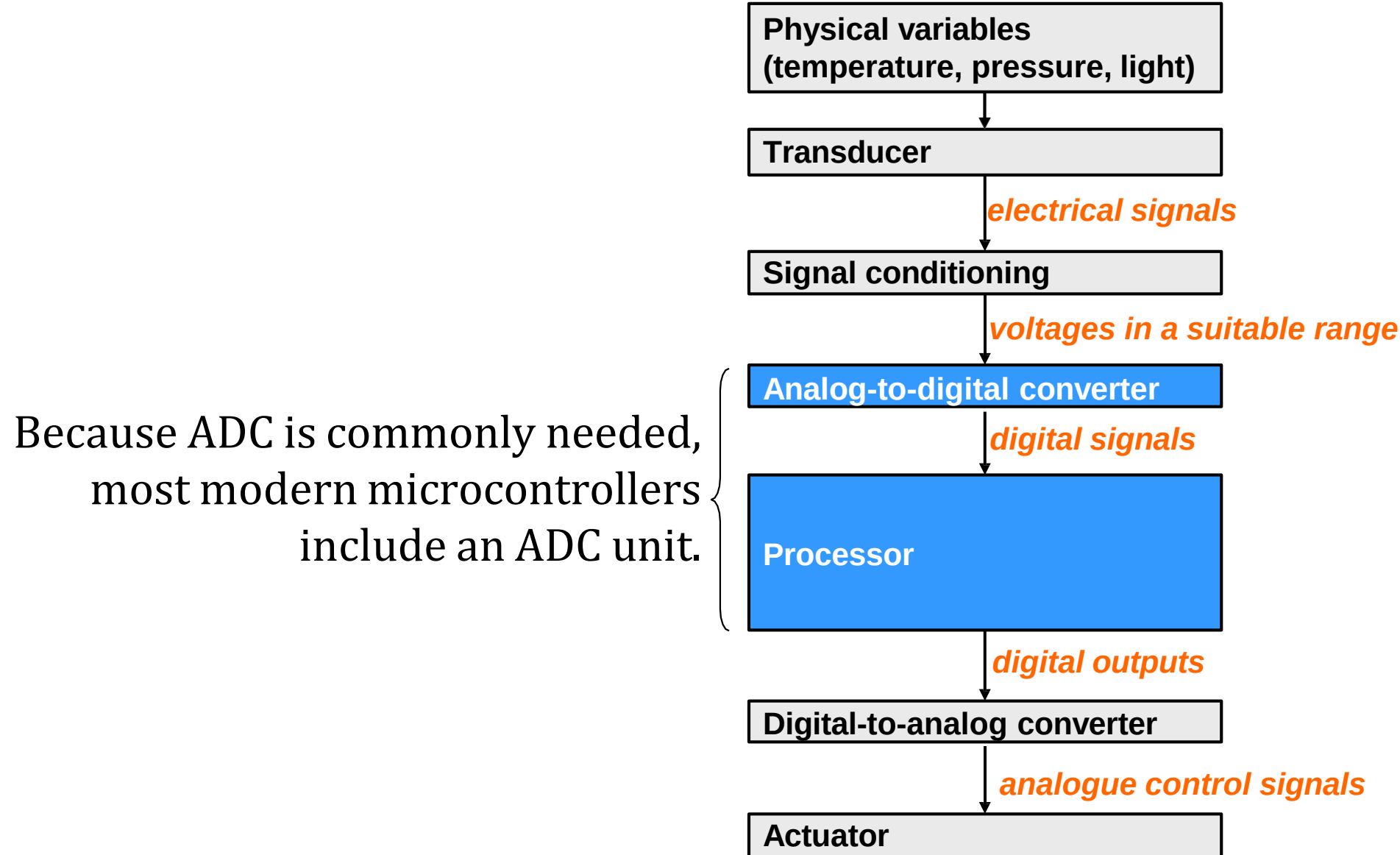


ADC digital value = 417,	input voltage = 2036 (mV)
ADC digital value = 418,	input voltage = 2041 (mV)
ADC digital value = 419,	input voltage = 2045 (mV)
ADC digital value = 416,	input voltage = 2031 (mV)
ADC digital value = 415,	input voltage = 2026 (mV)
ADC digital value = 410,	input voltage = 2001 (mV)
ADC digital value = 410,	input voltage = 2001 (mV)
ADC digital value = 407,	input voltage = 1987 (mV)
ADC digital value = 407,	input voltage = 1987 (mV)
ADC digital value = 408,	input voltage = 1992 (mV)
ADC digital value = 415,	input voltage = 2026 (mV)
ADC digital value = 414,	input voltage = _

UNIVERSITY OF WOLLONGONG AUSTRALIA

Using ADC to measure the lighting level

A-to-D conversion: Typical embedded application



A-to-D conversion: Example applications

■ Local positioning sensor for object tracking

Measure the distance b/t FM transmitter and receiver.

- The receiver has a Receiver Signal Strength Indicator (RSSI).
- The RSSI voltage is inversely proportional to the squared distance.



■ Temperature sensor for shower water

Measure the temperature of the shower water.

- Use a thermistor as a sensor.
- Control hot/cold water valves.



A-to-D conversion: Example applications

■ Obstacle sensor & audio cues in the cane for the blind

- Measure distance to nearest object with an ultrasonic sensor.
- The sensor output is digitized using the ADC.



■ Electric fence monitoring

- Determine if an electric fence is being tampered.
- Measure the voltage levels of an electric fence.



A-to-D conversion: Example applications

■ Wireless irrigation system

- Measure soil moisture using a resistor and an ADC.
- Turn ON or OFF the sprinklers.
- Transmit data wirelessly to the base station.



■ Intelligent clothesline

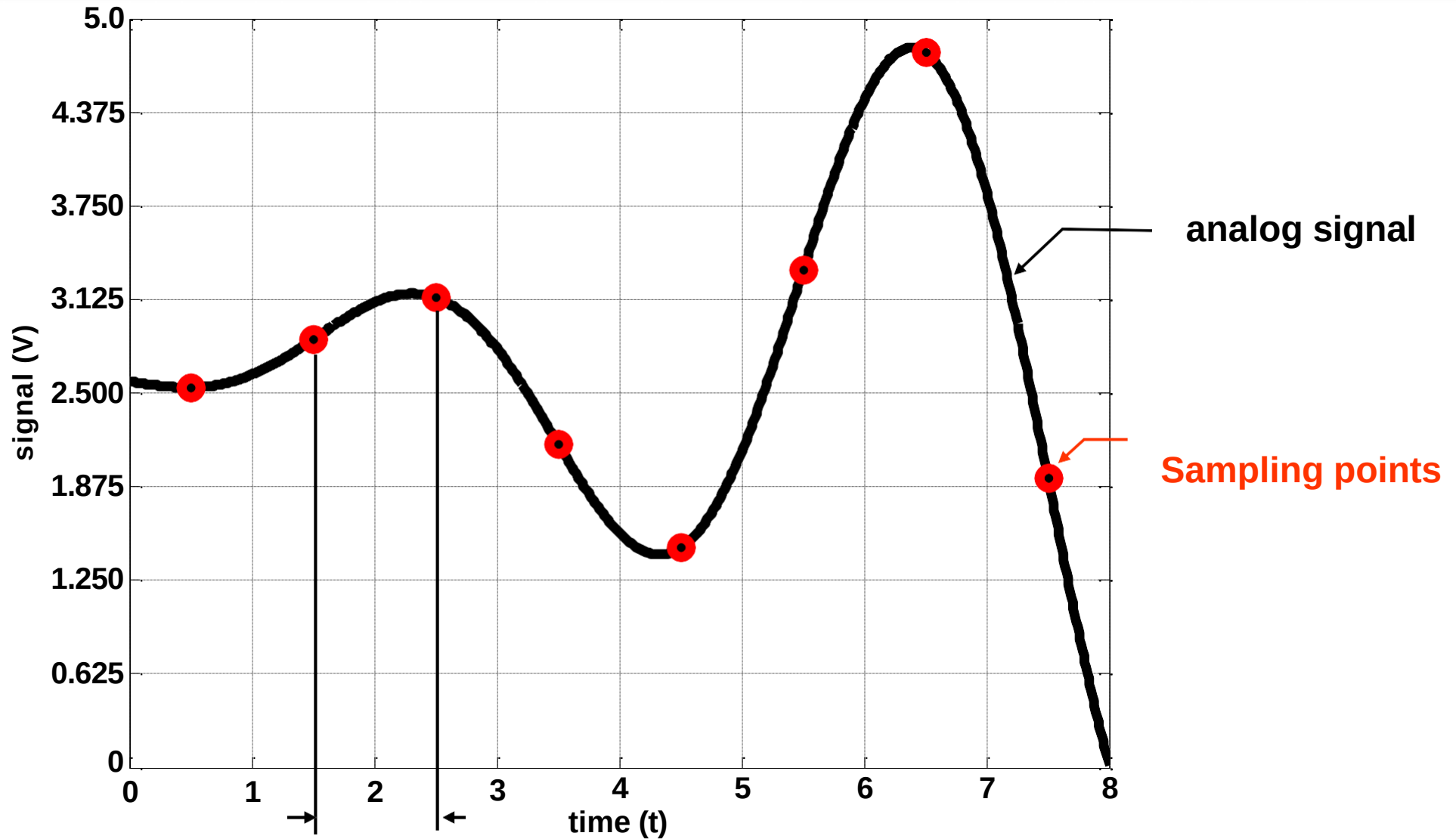
- Use sensors to measure humidity, temperature, and wind speed.
- Open/close the clothesline cover to protect against rain.



A-to-D conversion: The process

- There are two related steps in A-to-D conversion: sampling & quantisation.
- Sampling:
 - The analog signal is extracted, usually at regular times.
 - The samples have real values.
- Quantisation:
 - The samples are quantised to discrete levels.
 - Each sample is represented as a digital value.

Sampling an analog signal



sampling period T_s

➡ What is a suitable sampling period for a signal?

The sampling theorem

An analogue signal $x(t)$ with frequencies of no more than F_{max} can be reconstructed exactly from its samples if the sampling rate satisfies:

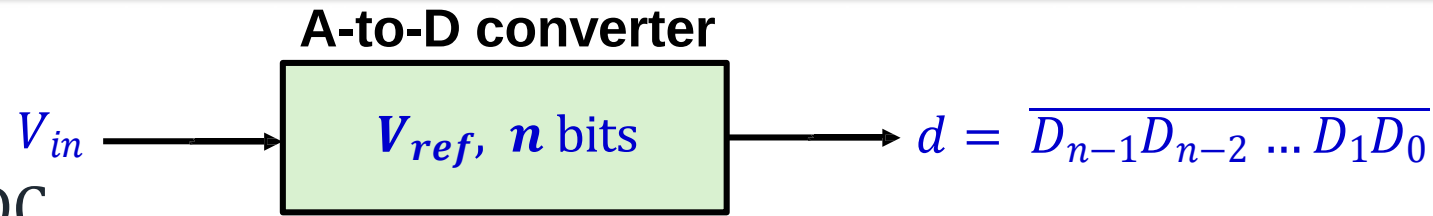
$$F_s \geq 2 \times F_{max}$$

- **Significance:** If maximum frequency of the signal is F_{max} , the sampling rate should be at least:

$$\text{Nyquist rate} = 2 \times F_{max}$$

- **Example:** If the maximum frequency of the signal is $F_{max} = 100 \text{ Hz}$, then the sampling rate should be at least 200 Hz.

Quantising the sampled signal



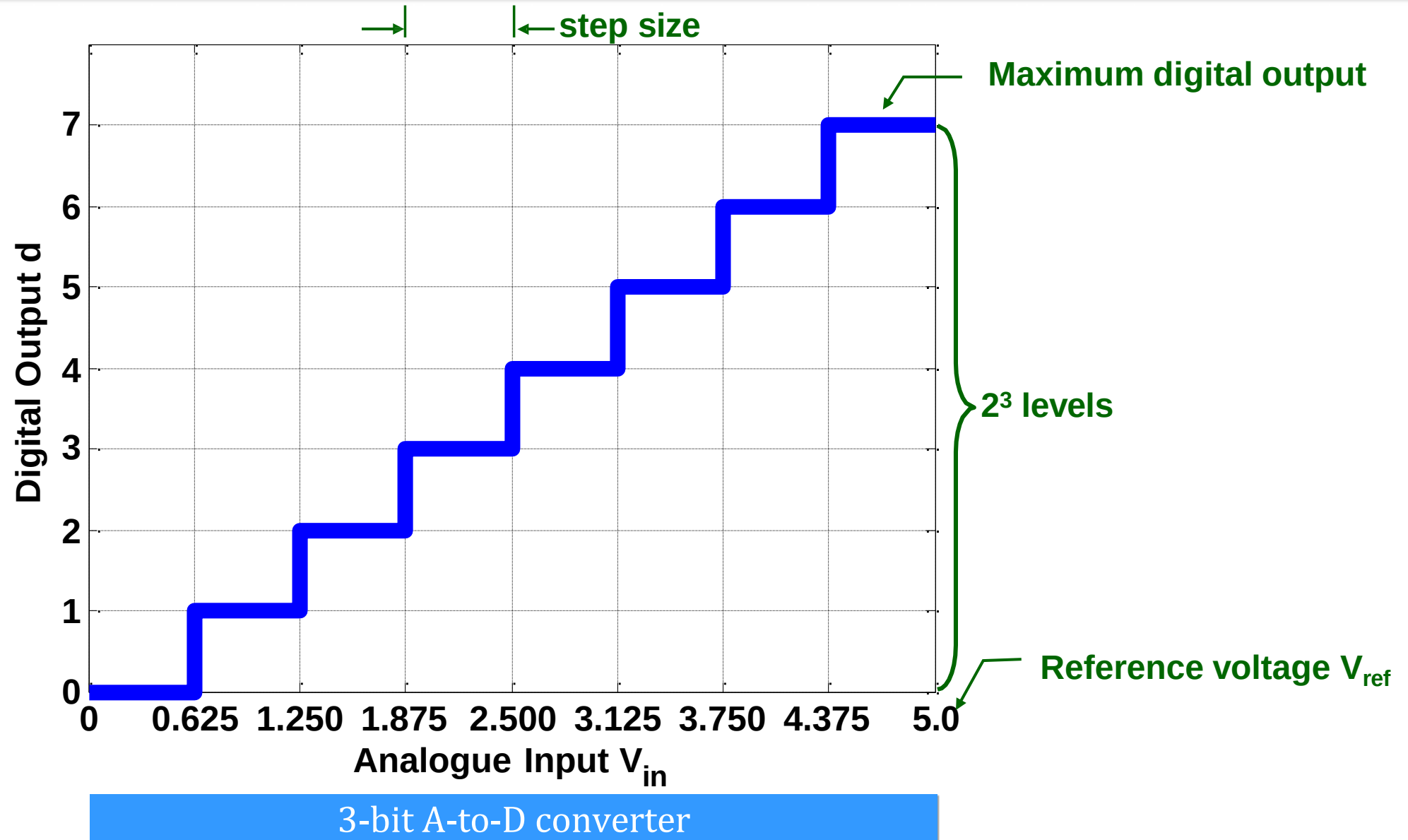
- Consider an n-bit ADC.
- Let V_{ref} be the **reference voltage**.
- Let V_{in} be the analogue input voltage.
- Let V_{min} be the minimum allowable input voltage, usually $V_{min} = 0$.
- The **digital output**, $d = \overline{D_{n-1}D_{n-2} \dots D_1D_0}$, is given as

$$d = \text{round down} \left\lfloor \frac{V_{in} - V_{min}}{\text{step size}} \right\rfloor$$

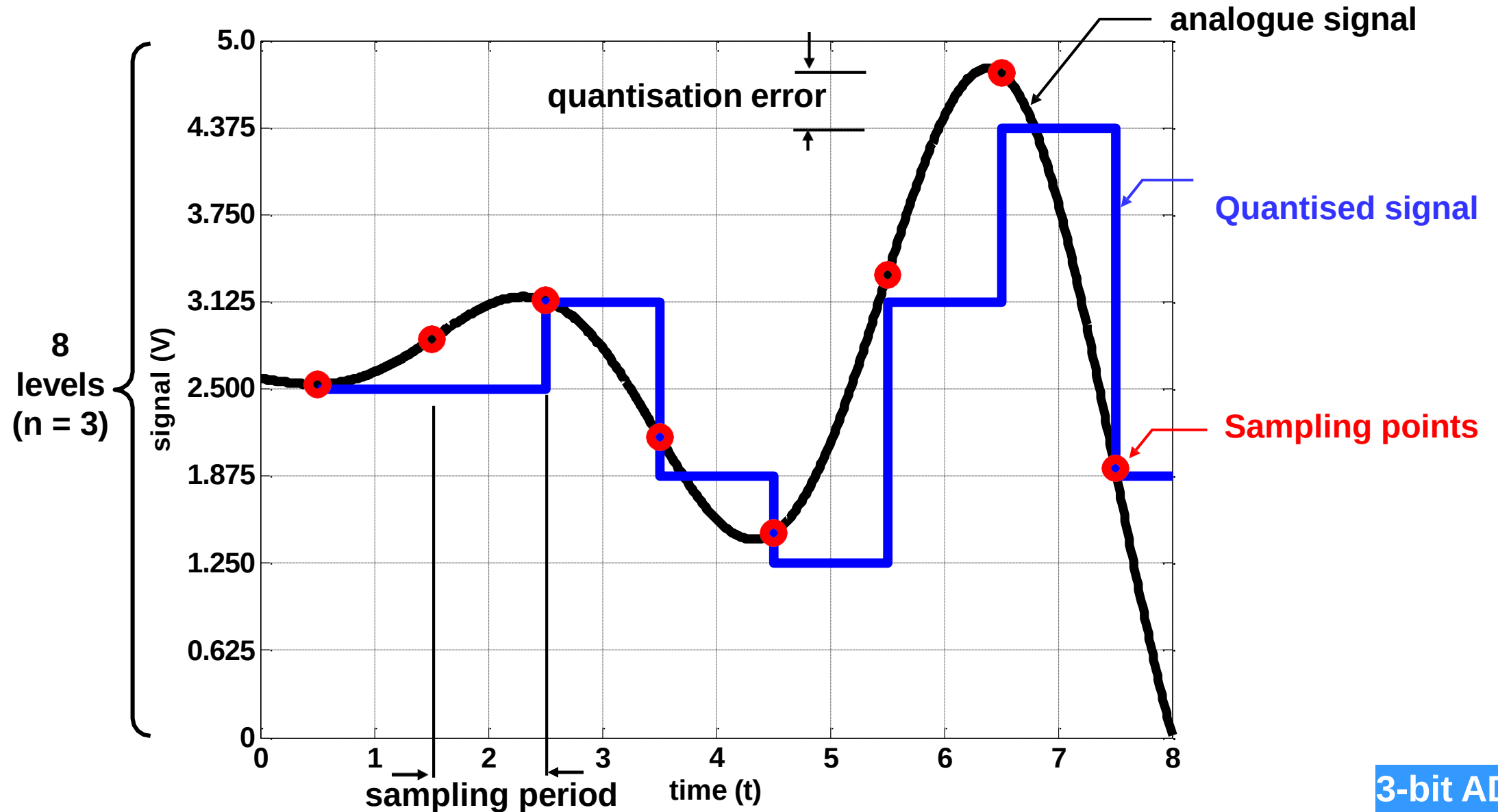
- The **step size** (resolution) is the smallest change in input that can be discerned by the ADC:

$$\text{step size} = \frac{V_{ref} - V_{min}}{2^n} = \frac{V_{max} - V_{min}}{2^n - 1}$$

Quantising the sampled signal



Quantising the sampled signal



3-bit ADC

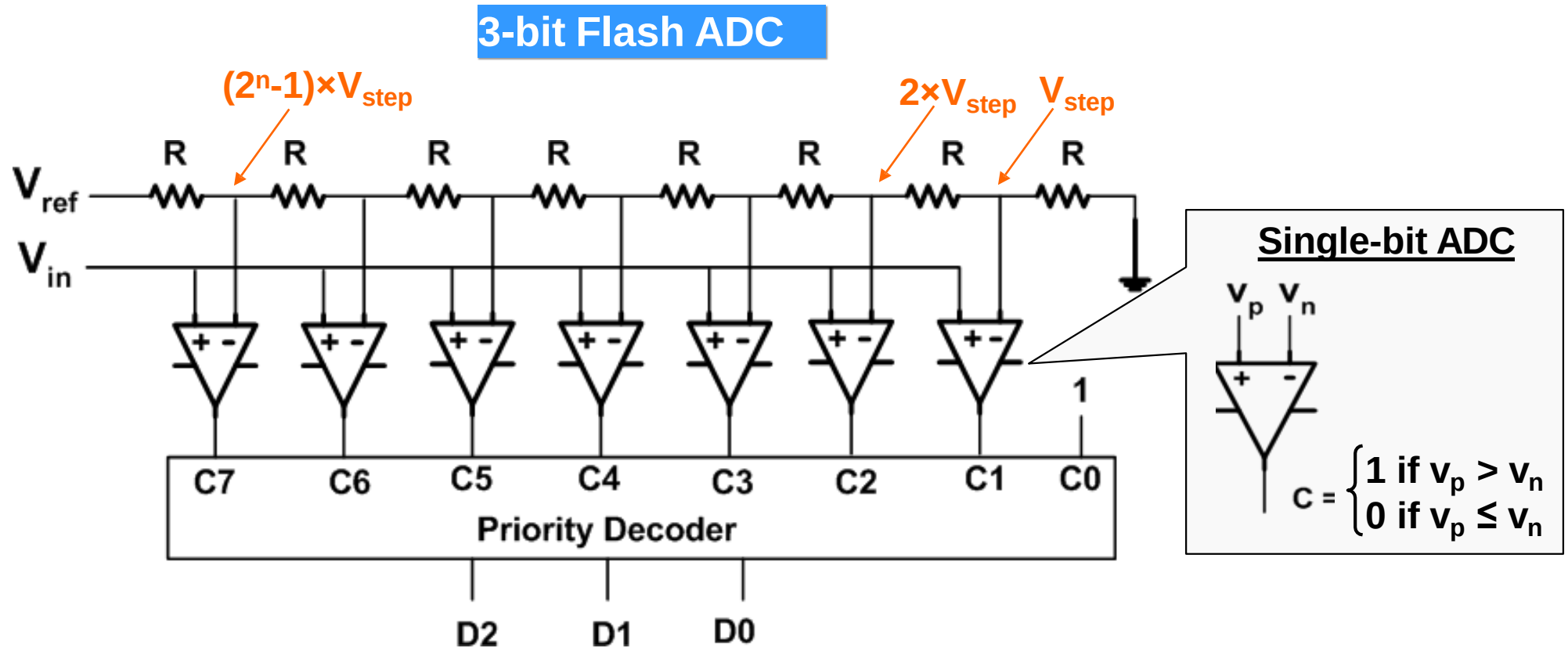
A-to-D converter: Parameters

- **Number of bits n :** The higher is the number of bits, the more precise is the digital output.
- **Quantization error E_q :** The average difference between the analog input and the quantised value. For an ideal ADC, we have $E_q = \frac{1}{2} \text{ step size}$.
- **Sample time T_{sample} :** A sampling capacitor must be charged for a duration of T_{sample} before conversion taking place.
- **Conversion time T_{conv} :** Time taken to convert the voltage on the sampling capacitor to a digital output.

A-to-D converter: Designs

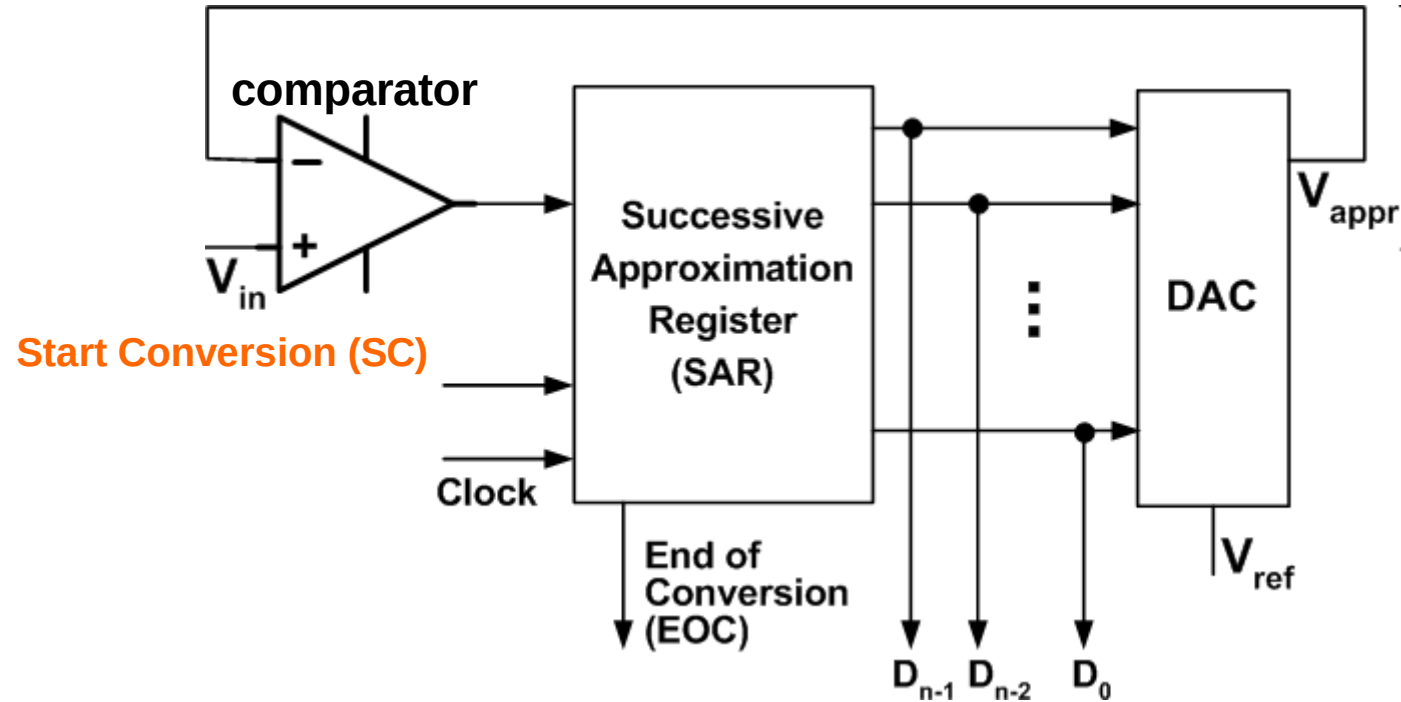
- There are many designs for analog-to-digital converters.
- We'll consider two common designs.
 - Flash ADC.
 - Successive-approximation ADC.

Flash ADC



- An n-bit flash ADC uses (2^n-1) comparators, and a priority decoder.
- It is the fastest type of ADC.
- However, it has a limited resolution, is expensive, and requires large power.

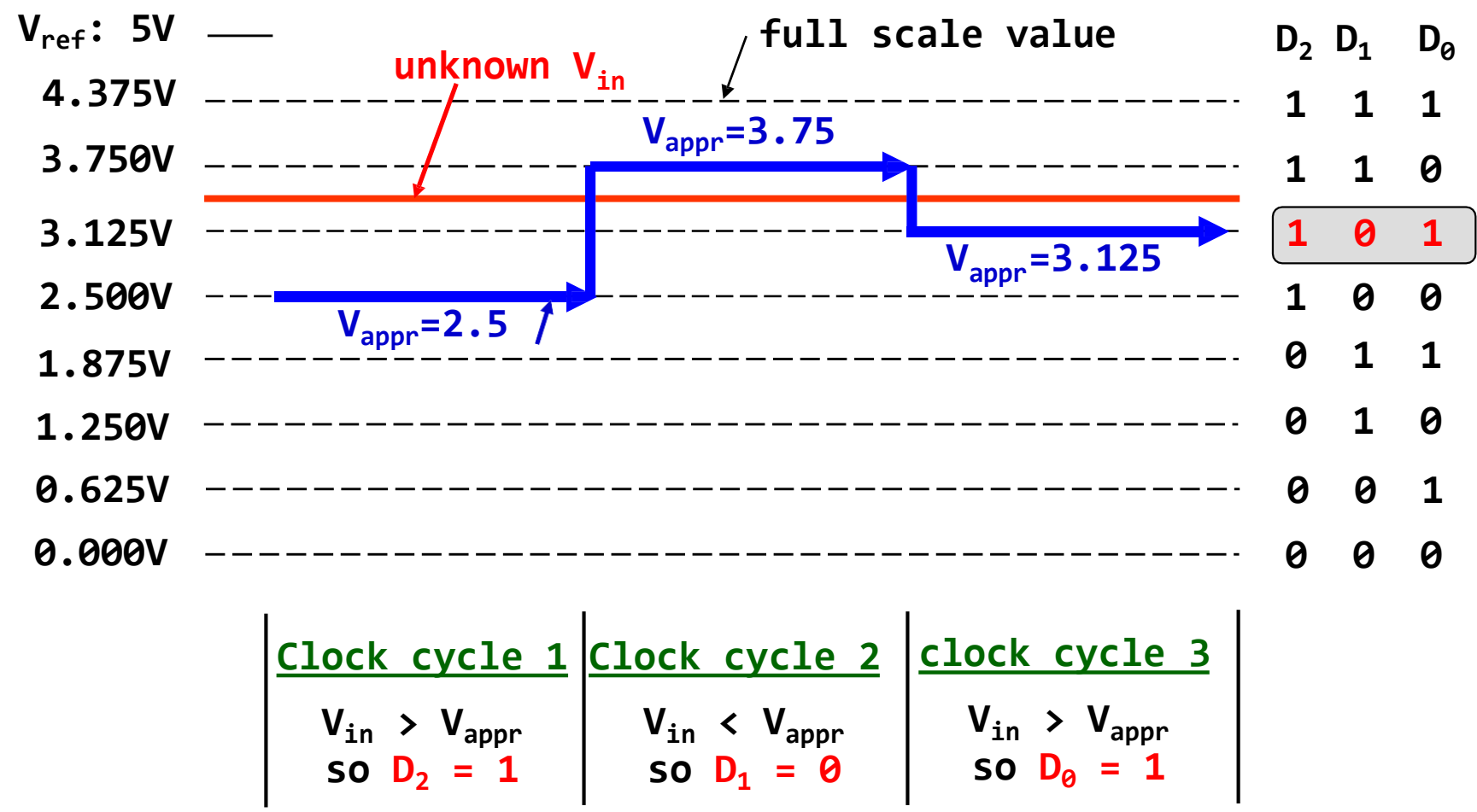
Successive-approximation ADC



- A DAC is used to generate approximations of the input voltage.
- A comparator is used to compare V_{in} and V_{appr} .
- In each cycle, SAR finds one output bit using the comparator.
- To start conversion, set $SC = 1$. When conversion ends, signal $EOC = 1$.
- A quite fast, widely-used ADC design.

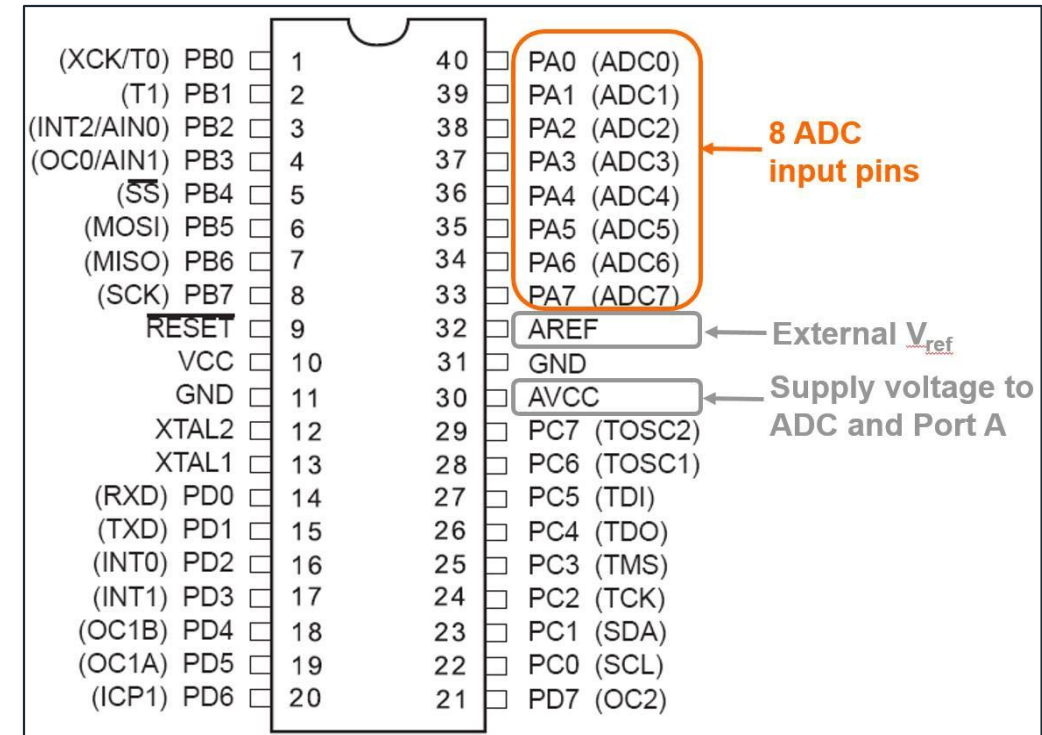
Successive-approximation ADC

Binary search for a 3-bit ADC



The ADC in ATmega16

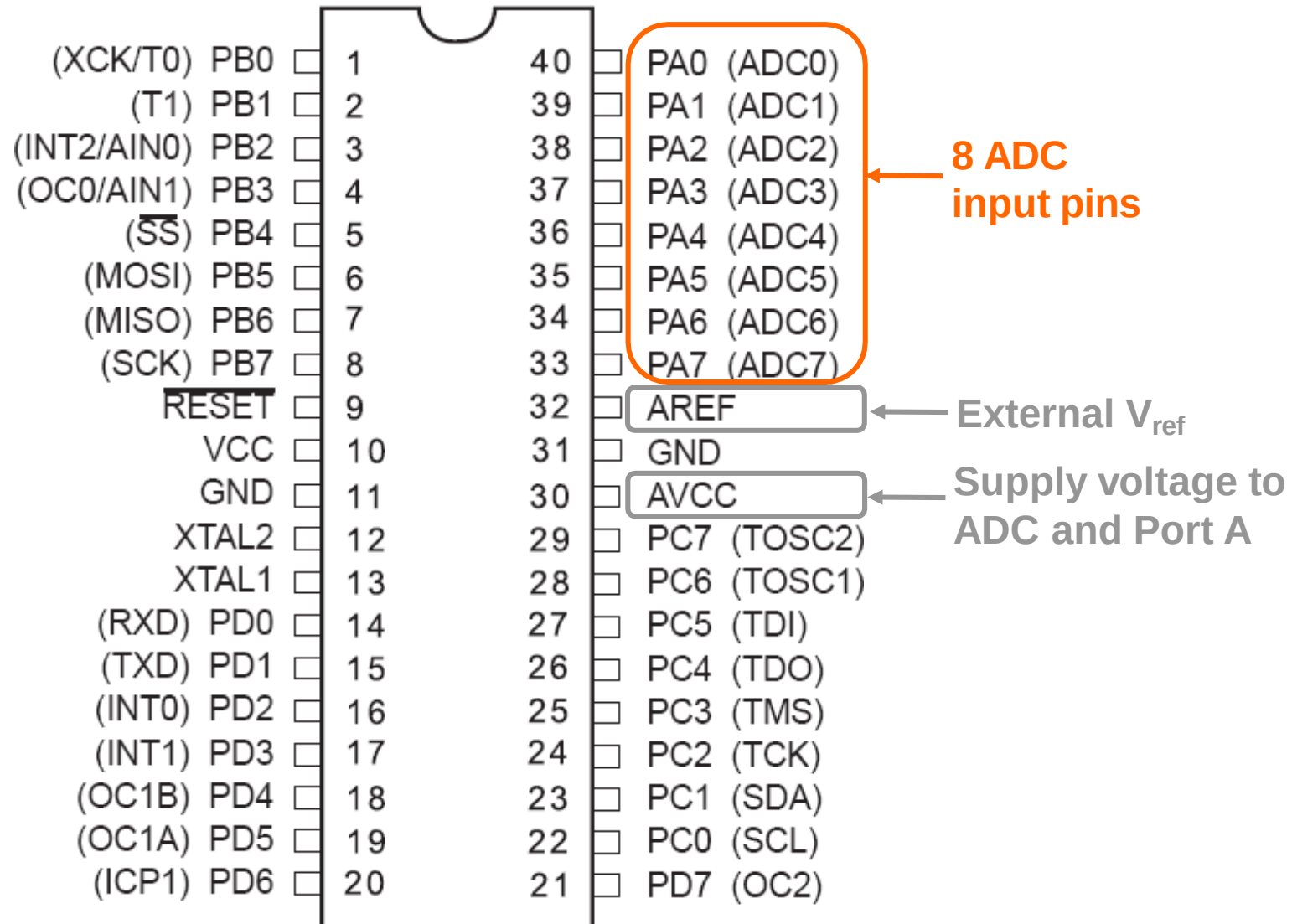
- The ADC in ATmega16 has a 10-bit resolution.
 - The digital output has $n = 10$ bits.
- The ADC has 8 input channels.
 - Analogue input can come from 8 different sources.
 - However, it performs conversion on only one channel at a time.



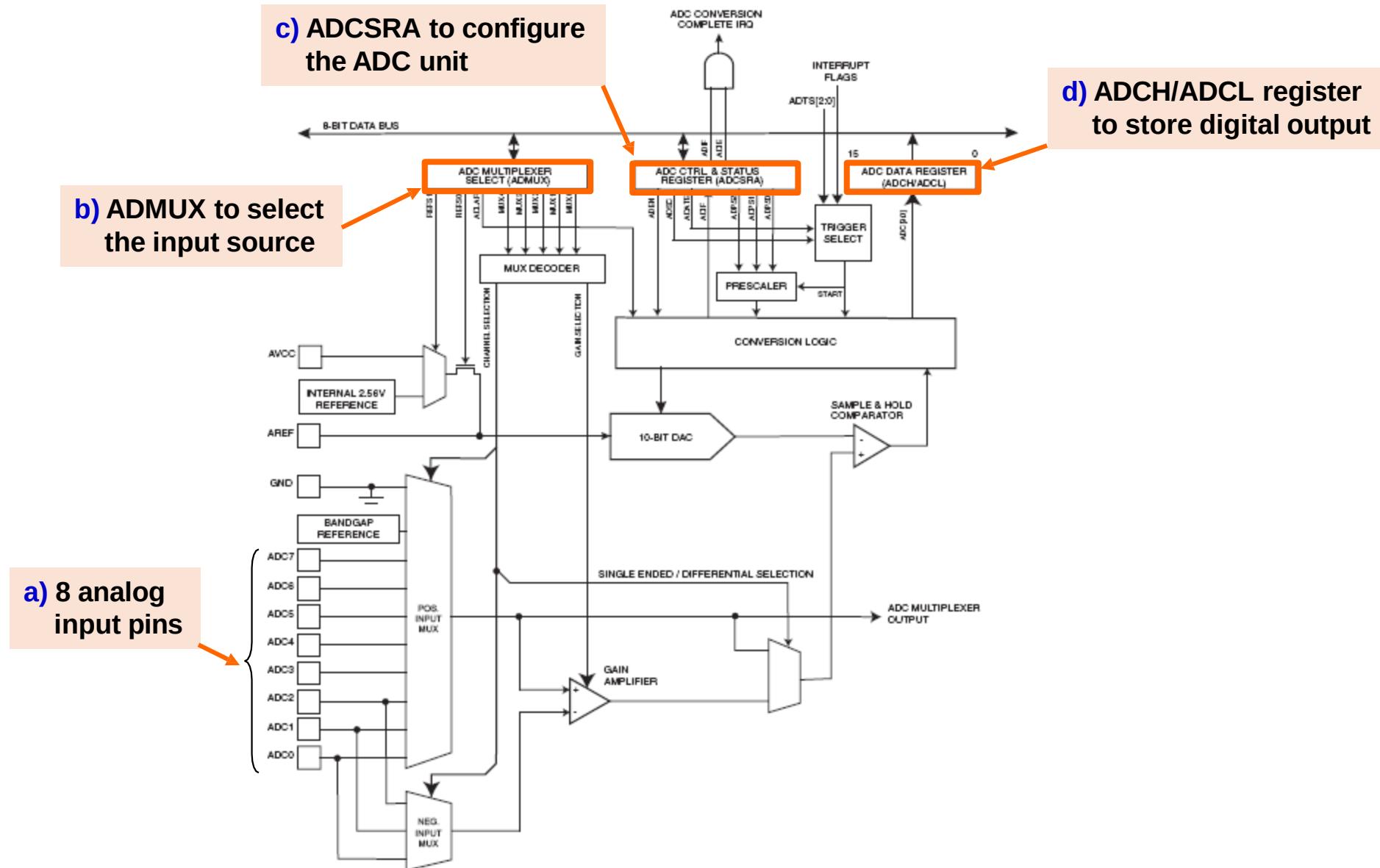
The ADC in ATmega16

- If default reference voltage $V_{\text{ref}} = 5\text{V}$ is used.
 - step size: $5(\text{V})/1024 \text{ (steps)} = 4.88\text{mV}$.
 - error range: $= \pm 4.88\text{mV}$.
- The clock rate of the ADC can be different from the CPU clock rate.
 - One ADC conversion takes 13 ADC clock cycles.
 - An ADC prescaler will decide the actual ADC clock cycle.

ADC unit: Relevant pins



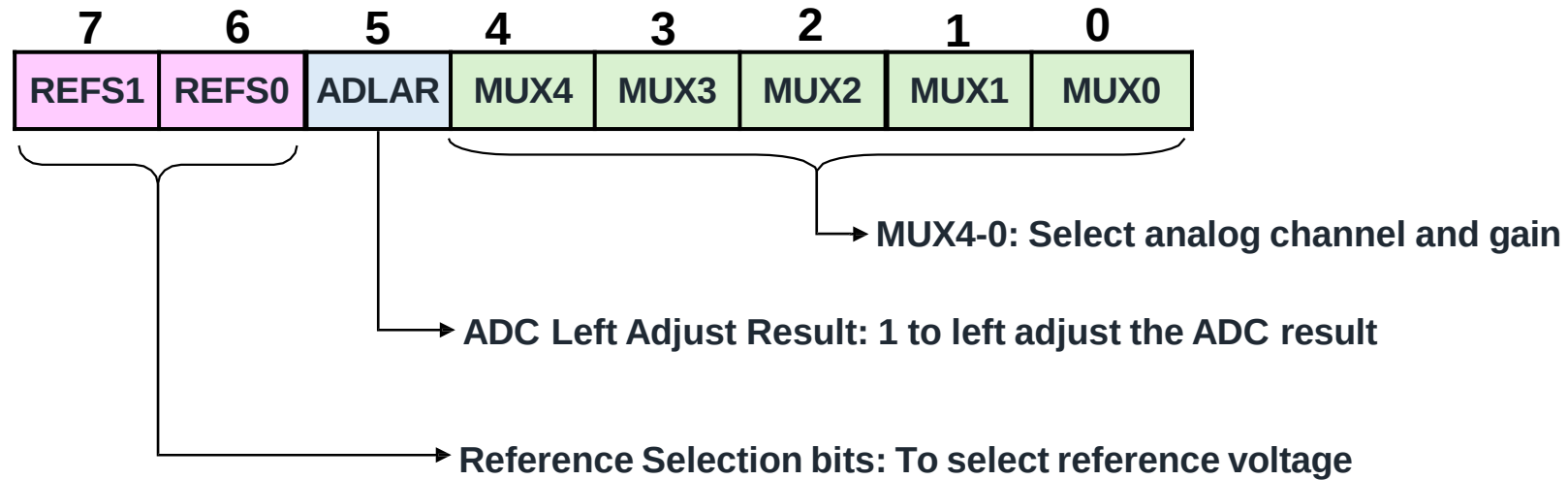
ADC unit: Block diagram



We study three major aspects of the ADC unit.

1. What are the relevant ADC registers?
 - a) ADMUX
 - b) ADCH/ADCL
 - c) ADCSRA
 - d) SFIOR
2. What are the steps to use the ADC?
3. How to use the ADC interrupt?

ADC Multiplexer Selection Register (ADMUX)



- Reference voltage V_{ref} can be selected among 3 choices.
- Analogue input can be selected among 8 pins, with optional differential input and gain.
- ADLAR flag determines how the 10-bit digital output is stored in output registers.

Selecting reference voltage V_{ref}

ADC reference voltage selection.



REFS1	REFS0	Voltage Reference Selection
0	0	AREF, Internal V_{ref} turned off
0	1	AVCC with external capacitor at AREF pin
1	0	Reserved
1	1	Internal 2.56V voltage reference with external capacitor at AREF pin

- Usually, mode 01 is used: AVCC = 5V as reference voltage.
- If input voltage has a larger range, we can use mode 00 to select an external reference voltage.

Selecting input source and gain factor

MUX4..0	Single Ended Input	Positive Differential Input	Negative Differential Input	Gain
00000	ADC0	N/A		
00001	ADC1			
00010	ADC2			
00011	ADC3			
00100	ADC4			
00101	ADC5			
00110	ADC6			
00111	ADC7			
01000	N/A	ADC0	ADC0	10x
01001		ADC1	ADC0	10x
01010 ^(f)		ADC0	ADC0	200x
01011 ^(f)		ADC1	ADC0	200x
01100		ADC2	ADC2	10x
01101		ADC3	ADC2	10x
01110 ^(f)		ADC2	ADC2	200x
01111 ^(f)		ADC3	ADC2	200x
10000		ADC0	ADC1	1x
10001		ADC1	ADC1	1x
10010		ADC2	ADC1	1x
10011		ADC3	ADC1	1x
10100		ADC4	ADC1	1x
10101		ADC5	ADC1	1x
10110		ADC6	ADC1	1x
10111		ADC7	ADC1	1x
11000		ADC0	ADC2	1x
11001		ADC1	ADC2	1x
11010		ADC2	ADC2	1x
11011		ADC3	ADC2	1x
11100		ADC4	ADC2	1x

ADC input source.

Choices of analogue input voltage:

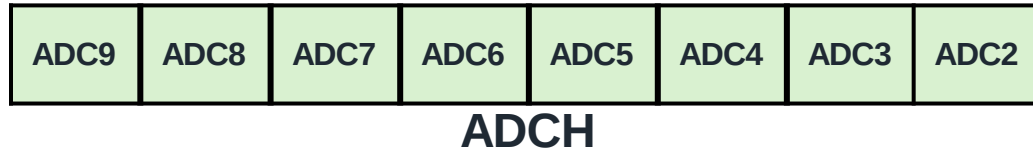
- 8 ADC pins (ADC0 to ADC7).
- Difference between 2 ADC pins.
- Different gain factors (1, 10, 200).

AtMega328p

MUX4	MUX3	MUX2	MUX1	MUX0	Channel Selected
0	0	0	0	0	ADC0 (Pin A0)
0	0	0	0	1	ADC1 (Pin A1)
0	0	0	1	0	ADC2 (Pin A2)
0	0	0	1	1	ADC3 (Pin A3)
0	0	1	0	0	ADC4 (Pin A4)
0	0	1	0	1	ADC5 (Pin A5)
0	0	1	1	0	ADC6 (Pin A6 - 32-pin package only)
0	0	1	1	1	ADC7 (Pin A7 - 32-pin package only)
0	1	0	0	0	Reserved
0	1	0	0	1	Reserved
0	1	0	1	0	Reserved
0	1	0	1	1	Reserved
0	1	1	0	0	Reserved
0	1	1	0	1	Reserved
0	1	1	1	0	Internal Temperature Sensor
0	1	1	1	1	Internal 1.1V Bandgap Reference

ADC Left Adjust flag and ADCH/L registers

When flag ADLAR = 1 (Left Aligned)



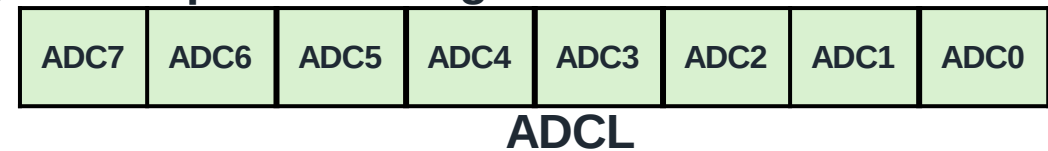
← 10 bits pushed to left



When flag ADLAR = 0 (Right Aligned)

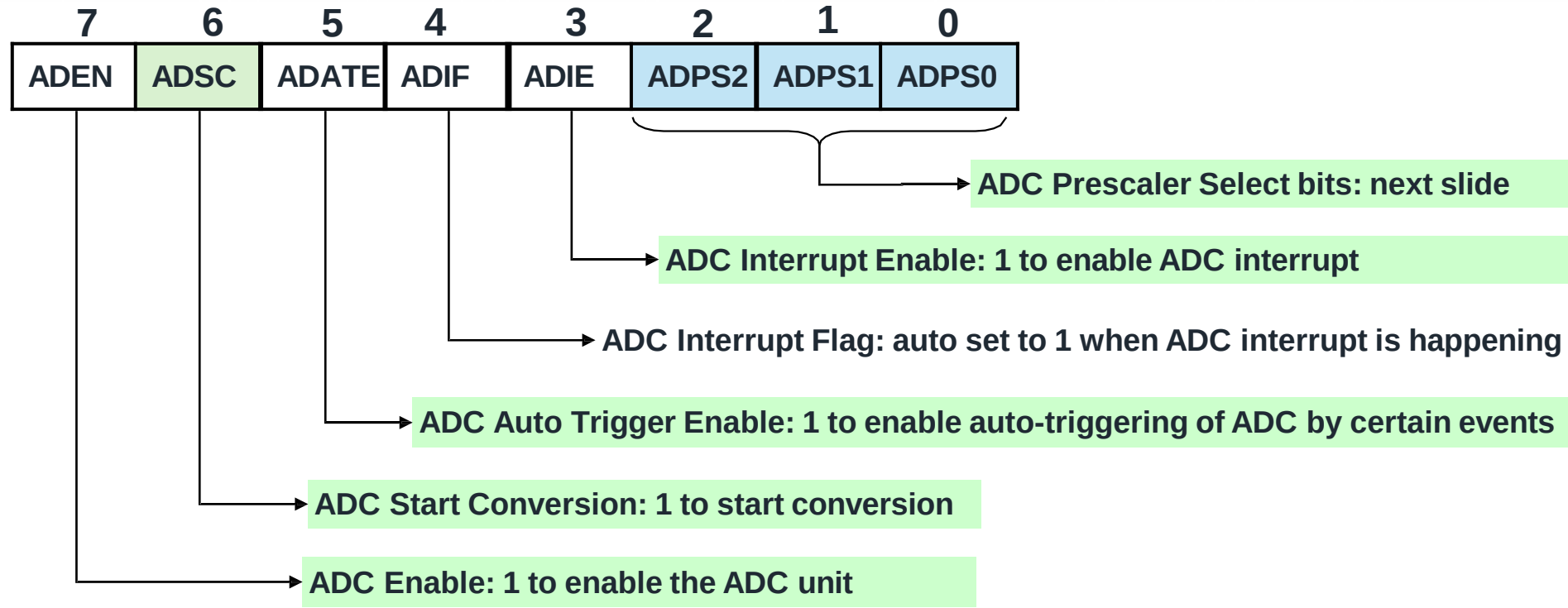


→ 10 bits pushed to right



- Digital output is stored in two 8-bit registers ADCH and ADCL.
- The format of ADCH and ADCL depends on flag ADLAR.
- When retrieving digital output, ADCL must be read before ADCH.

ADC Control and Status Register (ADCSRA)



- The ADC unit can operate in two modes: manual or auto-trigger.
- In manual mode, setting flag $ADSC = 1$ will start the conversion.
When conversion is completed, flag $ADSC = 0$ automatically.
- In auto-trigger mode, a predefined event will start the conversion.

ADC clock

ADC Prescaler Selection.



ADPS2	ADPS1	ADPS0	Division Factor
0	0	0	2
0	0	1	2
0	1	0	4
0	1	1	8
1	0	0	16
1	0	1	32
1	1	0	64
1	1	1	128

- The ADC clock is obtained by dividing the CPU clock versus a division factor.
- There are 8 possible division factors, selected by 3 flags: {ADPS2, ADPS1, ADPS0}
- Example: Using internal clock of 1Mz, and ADC prescaler bits of '011'

One ADC clock cycle = $8 \times \text{CPU clock cycle} = 8 \mu\text{s}$.

Special Function IO Register (SFIOR)

ADC Auto Trigger Source

ADTS2	ADTS1	ADTS0	-	ACME	PUD	PSR2	PSR10
-------	-------	-------	---	------	-----	------	-------

ADC Auto Trigger Source.

ADTS2	ADTS1	ADTS0	Trigger Source
0	0	0	Free Running mode
0	0	1	Analog Comparator
0	1	0	External Interrupt Request
0	1	1	Timer0 Compare Match
1	0	0	Timer0 Overflow
1	0	1	Timer1 Compare Match B
1	1	0	Timer1 Overflow
1	1	1	Timer1 Capture Event

Three flags in register SFIOR specify the event that will auto-trigger an A-to-D conversion.

Steps to use the ADC

- **Step 1:** Configure the ADC using registers ADMUX, ADCSRA, SFIOR.
 - What is the ADC source?
 - What reference voltage to use?
 - Align left or right the result in {ADCH, ADCL}?
 - Enable or disable ADC auto-trigger?
 - Enable or disable ADC interrupt?
 - What is the ADC prescaler?
- **Step 2:** Start ADC operation by writing to flag ADSC (register ADCSRA).
- **Step 3:** Extract ADC result
 - Wait until flag ADSC becomes 0.
 - Read the result from register ADCL and then ADCH.

Performing ADC

Write C code to repeatedly perform ADC on a sinusoidal input and display the result on LEDs.

■ Step 1: Configure the ADC

- What is the ADC source? ADC1 (pin A.1)
- What reference voltage to use? AVCC = 5V
- Align left or right? Left (top 8 bits in ADCH)
- Enable or disable ADC auto-trigger? Disable
- Enable or disable ADC interrupt? Disable
- What is the ADC pre-scaler? 2 (fastest, selector 001)

REFS1	REFS0	ADLAR	MUX4	MUX3	MUX2	MUX1	MUX0	ADMUX
0	1	1	0	0	0	0	1	
ADEN	ADSC	ADATE	ADIF	ADIE	ADPS2	ADPS1	ADPS0	ADCSRA
1	0	0	0	0	0	0	1	

■ Steps 2 and 3: Show next in the C program.

Example : adc.c

```
#include<avr/io.h>
int main (void){
    unsigned char result;

    DDRB = 0xFF;          // set port B for output
```

REFS1	REFS0	ADLAR	MUX4	MUX3	MUX2	MUX1	MUX0	ADMUX
0	1	1	0	0	0	0	1	
ADEN	ADSC	ADATE	ADIF	ADIE	ADPS2	ADPS1	ADPS0	ADCSRA
1	0	0	0	1	0	0	1	

```
    // Configure the ADC module of the ATmega16
    ADMUX = 0b01100001; // REFS1:0 = 01    -> AVCC = 5V as reference,
                        // ADLAR   = 1      -> Left adjust,
                        // MUX4:0  = 00001 -> ADC1 as input,
```

1

```
    ADCSRA = 0b10000001; // ADEN = 1:  enable ADC,
                        // ADSC = 0:  don't start conversion yet,
                        // ADATE = 0:  disable auto trigger,
                        // ADIE  = 0:  disable ADC interrupt,
                        // ASPS2:0 = 001: ADC prescaler = 2.
```

```
    while(1){          // main loop
```

```
        // Start conversion by setting flag ADSC
        ADCSRA = ADCSRA | 0b01000000;
```

2

```
        // Wait until conversion is completed
        while (ADCSRA & 0b01000000){;}
```

3

```
        // Read the top 8 bits, output to PORTB
        result = ADCL; result = ADCH;
```

```
        PORTB = ~result; // Port B connected to LEDs (0 = ON, 1 = OFF)
```

```
    }
    return 0;
```

Using ADC interrupt

- In the previous polling approach, we must check the ADSC flag to know when an ADC operation is completed.
- Alternatively, the ADC unit can trigger an interrupt when ADC is done.
- We enable ADC interrupt through ADIE flag in register ADCSRA.
- In the ISR, we write code to read registers ADCL and ADCH.
- ADC interrupt is usually combined with auto-trigger mode.



Example : ADC interrupt

Write interrupt-driven program to digitise a sinusoidal signal and display the result on LEDs.

- Step 1: Configure the ADC.
 - What is the ADC source? ADC1
 - What reference voltage to use? AVCC = 5V
 - Align left or right? Left (top 8 bits are in ADCH)
 - Enable or disable ADC auto-trigger? Disable
 - Enable or disable ADC interrupt? Enable
 - What is the prescaler? 2 (fastest conversion)
- Step 2: Start ADC operation (set ADSC flag = 1).
- Step 3: In ISR, read and store ADC result in a global variable.

Example : Program adc_int.c

```
#include<avr/io.h>
#include<avr/interrupt.h>
```

```
volatile unsigned char result;
```

```
ISR(ADC_vect){
    result = ADCH; // Read the top 8 bits, and store in global variable
}
```

```
int main (void){
    DDRB = 0xFF; // set port B for output

    // Configure the ADC module of the ATmega16
    ADMUX = 0b01100001; // REFS1:0 = 01 -> AVCC as reference,
                        // ADLAR = 1 -> Left adjust
                        // MUX4:0 = 00001 -> ADC1 as input
```

```
    ADCSRA = 0b10001001; // ADEN = 1: enable ADC,
                        // ADSC = 0: don't start conversion yet
                        // ADATE = 0: disable auto trigger,
                        // ADIE = 1: enable ADC interrupt
                        // ASPS2:0 = 001: prescaler = 2
```

```
    sei(); // enable interrupt system globally
    while(1){ // main loop
```

```
        ADCSRA = ADCSRA | 0b01000000; // start a conversion
```

```
        PORTB = ~result;
```

```
    }
    return 0;
```

```
}
```

REFS1	REFS0	ADLAR	MUX4	MUX3	MUX2	MUX1	MUX0	ADMUX
0	1	1	0	0	0	0	1	
ADEN	ADSC	ADATE	ADIF	ADIE	ADPS2	ADPS1	ADPS0	ADCSRA
1	0	0	0	1	0	0	1	

3

1

2

Example application of the ADC

- We study an application of the ADC in the ATmega16.
- A joystick is used to move a camera up/down/left/right, as in a security console.
- The pan-tilt camera is mCAM100x.
- It can be moved by sending a character '4', '6', '8', or '2' via a serial connection: 9600bps, 8 data bits, 1 stop bit, no parity.



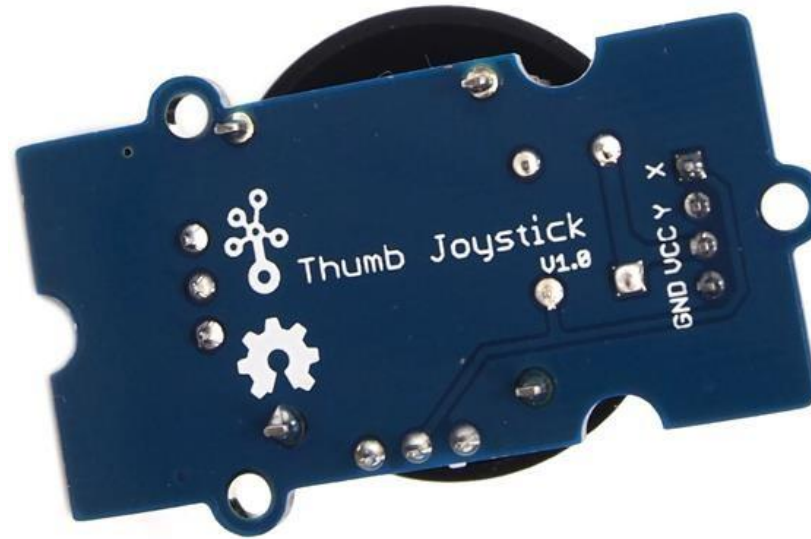
Joystick

- The joystick we use is Grove SS-COM90133P, by Seed Studio (www.seeedstudio.com).
- The joystick has 4 pins
 - GRD: ground
 - VCC: 5V-supply
 - X: x coordinate, analogue voltage between [0, 5V]
 - Y: y coordinate, analogue voltage between [0, 5V]



littlebirdelectronics.com/products/grove-thumb-joystick

Joystick



Description	Min	Typical	Max
Working voltage VCC	4.75V	5.00V	5.25V
X coordinate after digitising	206	516	798
Y coordinate after digitising	203	507	797

Joystick

Step 1: Configure the ADC

- What is the ADC source? ADC0 or ADC1
- What reference voltage to use? AVCC = 5V
- Align left or right? Right (bottom 8 bits in ADCL)
- Enable or disable ADC auto-trigger? Disable
- Enable or disable ADC interrupt? Disable
- What is the ADC pre-scaler? 128 (slowest conversion)

REFS1	REFS0	ADLAR	MUX4	MUX3	MUX2	MUX1	MUX0	ADMUX
0	1	0	0	0	0	0	0/1	

ADEN	ADSC	ADATE	ADIF	ADIE	ADPS2	ADPS1	ADPS0	ADCSRA
1	0	0	0	0	1	1	1	

Steps 2 and 3: Show next in C program.

Joystick and Pan-Tilt Camera: Code

```
#include<avr/io.h>
int main (void){
    unsigned int result_low, result_high, result_x, result_y;
    // ... Initialise serial port ...

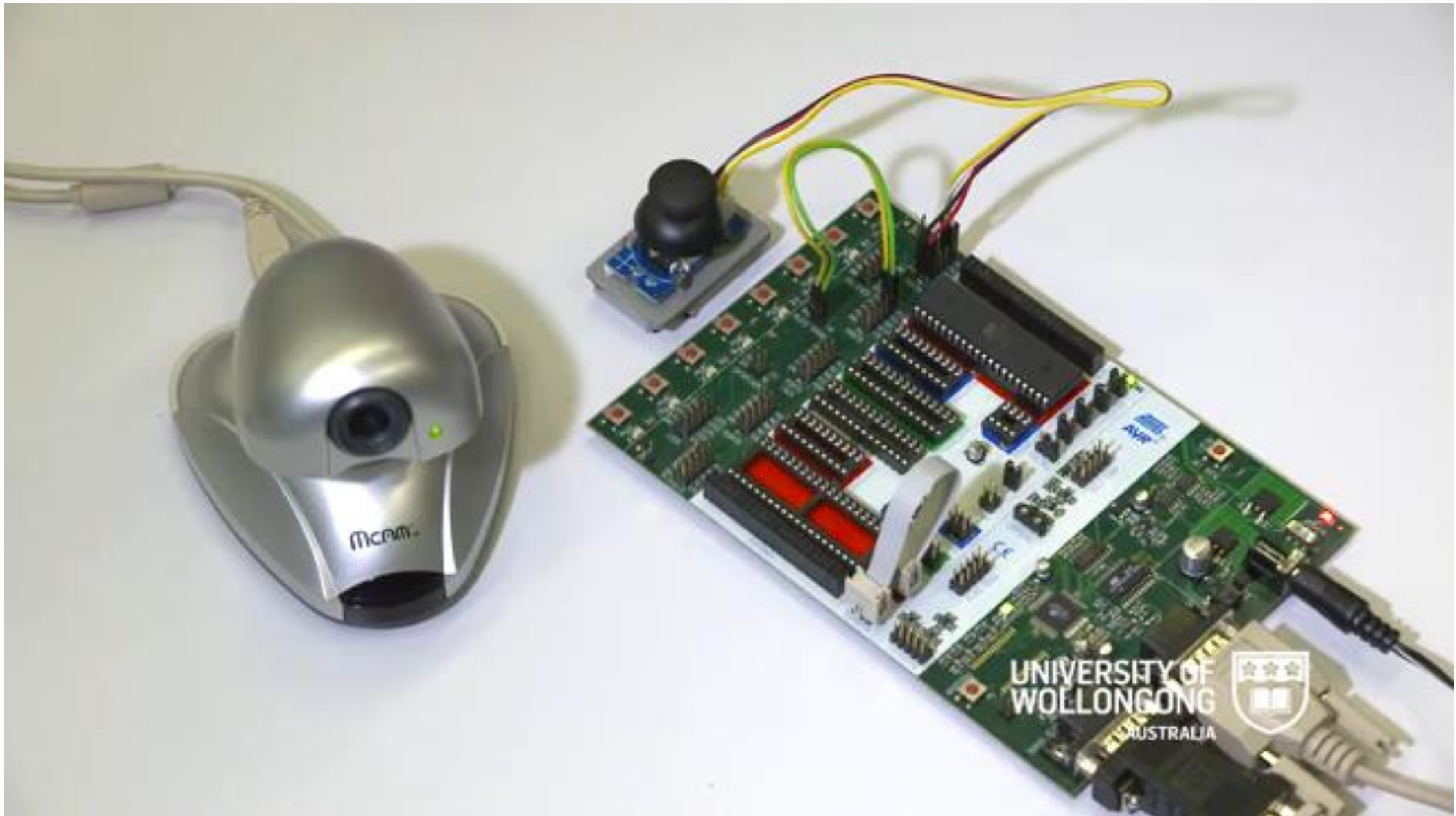
    // Configure the ADC module of the ATmega16
    ADCSRA = 0b10000111; // ADEN = 1: enable ADC,
                        // ADSC = 0: don't start conversion yet
                        // ADATE = 0: disable auto trigger,
                        // ADIE = 0: disable ADC interrupt
                        // ASPS2:0 = 111: prescaler = 128

    while(1){
        // Read X coordinate...
        ADMUX = 0b01000000; // REFS1:0 = 01    -> AVCC as reference,
                        // ADLAR = 0    -> Right adjust
                        // MUX4:0 = 00000 -> ADC0 as input
        ADCSRA = ADCSRA | 0b01000000; // Start conversion by setting flag ADSC
        while (ADCSRA & (1 << ADSC)){;} // Wait until conversion is completed
        // Read digital output
        result_low = ADCL; // low 8 bits in ADCL
        result_high = ADCH & 0b00000011; // bit 8 and 9 in ADCH
        result_x = (result_high << 8) + result_low;

        // Read Y coordinate...
        ADMUX = 0b01000001; // MUX4:0 = 00001 -> ADC1 as input
        ...

        // Serial port code for camera control ...
    }
    return 0;
}
```

Joystick and Pan-Tilt Camera: Demo



Lecture summary

■ Key points of this lecture

- Analogue-to-digital conversion process.
- Sampling and quantization steps.
- Using the ADC in the ATmega16 microcontroller.
- An example application of ADC.

Lecture references

- M. A. Mazidi, S. Naimi, S. Naimi, AVR Microcontroller and Embedded Systems: Using Assembly and C, 2nd edition Pearson, 2015. [[Chapter 13: ADC ...](#)]
- Atmel Corp., 8-bit AVR microcontroller with 16K Bytes In-System Programmable Flash ATmega16/ATmega16L, 2010. [[Analog to Digital Converter](#)]
- S. F. Barrett and D. J. Pack, Atmel AVR Microcontroller Primer: Programming and Interfacing, Morgan & Claypool Publishers, 2008. [[Chapter 3: Analog-to-Digital Conversion](#)]
- M. Mazidi, J. Mazidi, R. McKinlay, 8051 Microcontroller and Embedded Systems, Pearson New International, 2022.